

TrustBank CBS — Legacy → New Component Guide

Developer reference for the **strangler-fig migration** (ASP.NET WebForms / VB.NET → .NET 10 / C# MVC). For each **core reusable** legacy control it gives the new counterpart, a property mapping, a copy-paste usage recipe, and the key differences.

This file is the **source of truth**. The `.txt` / `.html` / `.pdf` siblings are generated from it by `docs/build-docs.mjs` — do not hand-edit them. The `component-docs-sync` subagent keeps all of these current.

Live demo: `/Dev/Components` (Development only) renders every reusable component below with dummy data and all options. Maintained by the `component-demo` agent.

How to read this

Where things live

- Legacy controls: `...\Trust.Bank.Publish\UserControls\*.ascx(.vb)` (+ paired `*.aspx` popups).
- New models: `Tf1Cbs.Framework\Models\{Shared,Search}\*.cs`
- New services / infra: `Tf1Cbs.Framework\{Services,Infrastructure}\*.cs`
- New partials: `Tf1Cbs.Host.Main\Views\Shared\*.cshtml`
- New client JS: `Tf1Cbs.Host.Main\wwwroot\js\*.js`

Four cross-cutting changes that touched every component

- The server owns the query.** Legacy shipped (encrypted) SQL in the popup URL; new screens never send SQL/columns/table to the client. The controller owns the query; only values cross the wire (`SearchRequest`).
- Opaque row ids.** Grid rows carry a `RowToken` (AES, bound to screen + session, single-use on write) instead of raw primary keys.
- Fail-closed access.** Every route is guarded by `CbsAccessMiddleware` (authenticated + module selected + URL in the user's menu) unless explicitly attributed.
- One in-page AJAX modal.** A single shared modal (`_SearchModal` + `cbs-search.js`) replaces per-control `window.showModalDialog` popups.

Index

Legacy control	New counterpart	Status
<code>TssiplDatePicker.ascx</code>	<code>_DatePicker</code> + <code>DatePickerModel</code> + <code>cbs-datepicker.js</code>	Available
<code>TssiplRecordSelector(.ascx)</code> V1/V2 + <code>RecordSelector.aspx</code>	<code>_RecordPicker</code> + <code>RecordPickerModel</code> + <code>cbs-search.js</code>	Available

Legacy control	New counterpart	Status
<code>TssiplSearchBox.ascx</code> + <code>RecordSelector.aspx</code> (grid)	<code>_SearchModal</code> + <code>SearchDescriptor</code> + <code>SearchEndpoint</code> + <code>RowToken</code>	Available
ad-hoc <code>alert()</code> / result popups	<code>Notify</code> + <code>_Notification</code> (ack modal) + <code>cbsNotify</code> (toast)	Available
<code>MainMenu.ascx</code> + <code>CollapsiblePanelStart/End.ascx</code>	<code>_MenuOverlay</code> / <code>_MenuGrid</code> + <code>MenuService</code>	Available
<code>ActionButtons.ascx</code>	<code>_ActionBar</code> + command-dispatch POST	Available
<code>Account.ascx</code> (3-segment composite) + <code>COA/Account TssiplRecordSelector</code> + <code>AccountSelector.aspx</code>	<code>_AccountPicker</code> (3 typed segment boxes) + <code>AccountPickerModel</code> + <code>_AccountSearchModal</code> + <code>cbs-account-picker.js</code> + <code>AccountService</code>	Available — selection plus opt-in <code>cbs:account-picked</code> / <code>cbs:account-cleared</code> host events (maker-checker deferred)

A. Date Picker

Calendar-free Day / Month / Year input.

At a glance

- Legacy: `UserControls\TssiplDatePicker.ascx(.vb)` — three dropdowns + a hidden value, server `ServerValidate` event, **no calendar**.
- New: `Views\Shared\_DatePicker.cshtml` · `Framework\Models\Shared\DatePickerModel.cs` · `wwwroot\js\cbs-datepicker.js`.

Mapping

Legacy	New	Notes
<code>uxDate</code> / <code>uxMonth</code> / <code>uxYear</code> dropdowns	three <code>select</code> (Day/Month/Year)	display-only (no name)
<code>uxHiddenValue</code>	hidden field named <code>Name</code>	posts ISO <code>yyyy-MM-dd</code> , binds to <code>DateTime?</code>
<code>RequiredField</code>	<code>[Required]</code> on the <code>DateTime?</code> form property	the picker has no <code>Required</code> prop — the selects post nothing, so required-ness belongs to the bound property
<code>ServerValidate</code> event	model binding + <code>DataAnnotations</code>	no event; standard MVC validation
client/server day fixups	<code>cbs-datepicker.js</code> leap-year day-clamp	Feb 29 / 30-day months handled

Legacy	New	Notes
(new) <code>MinYear / MaxYear</code>	year range, default <code>1900 – 2100</code>	ascending dropdown; also sets <code>min / max</code> on the calendar input
(new) <code>HideDay</code>	month/year-only mode	day pinned to the 1st at render, as legacy did
(none — legacy had no calendar)	 button → the browser's native picker	<code>showPicker()</code> on a collapsed <code><input type="date"></code> ; a pick fills the three dropdowns

Usage recipe

```
// Controller – seed the value (e.g. the working date for a new record)
vm.Form.ActionDate = session.WorkingDate?.WorkingDate ?? DateTime.Now.Date;
```

```
@* View – bind the hidden field to a DateTime? form property *@
@{
    var actionDate = new DatePickerModel
    {
        Name = "Form.ActionDate",
        Id = "Form_ActionDate",
        Value = Model.Form.ActionDate,
    };
}
<label class="req">Action Date</label>
<partial name="_DatePicker" model="actionDate" />
<span asp-validation-for="Form.ActionDate"></span>
```

```
// Form model – the bound target
public DateTime? ActionDate { get; set; }
```

Key differences / gotchas

- The three selects have no `name`; only the hidden ISO field posts. Always point `Name / Id` at the same `DateTime?`.
- Required-ness is `[Required]` on the form property plus `<label class="req">` for the visual cue; the picker itself has no flag for it.
- Year is a dropdown (type-ahead works); change `MinYear / MaxYear` per screen if the default range is wrong.
- The calendar is the **browser's own** (`showPicker()`), not a JS library — so it can't be restyled and looks a little different per browser. It needs Chrome/Edge 99+, Firefox 101+ or Safari 16+; anywhere else the button is inert and the three dropdowns still work.
- The dropdowns remain the accessible path (each has an `aria-label`); the calendar input is `tabindex="-1"` and `aria-hidden`, i.e. a mouse convenience layered on top.

B. Record Picker

Single-value lookup: a text box with Go / Pick / Clear, backed by the shared search modal.

At a glance

- Legacy: `UserControls\TssiplRecordSelector.ascx(.vb)` (V1) and `TssiplRecordSelectorV2.ascx(.vb)` + `RecordSelector(V2).aspx` popup.
- New: `Views\Shared\_RecordPicker.cshtml` · `Framework\Models\Shared\RecordPickerModel.cs` · `Framework\Models\Search\SearchDescriptor.cs` · `wwwroot\js\cbs-search.js`.

Mapping

Legacy property / event	New	Notes
<code>SelectQuery</code> (SQL in URL)	server-owned <code>Command</code> handler	no SQL on the wire
<code>DataValueField</code>	<code>PickValueKey</code> (+ <code>IdKey</code>)	column that fills the hidden id
<code>DataTextField</code>	<code>PickTextKey</code>	column that fills the display box
<code>VisibleColumns</code>	<code>Descriptor.Columns</code> + <code>Filters</code>	explicit allowlist
<code>HideGridColumns</code>	(whitelist <code>Columns</code> instead)	inverse approach
<code>ShowSearchFields</code>	presence of <code>Filters</code>	empty → single free-text box
<code>FillRecordsInitially</code>	<code>AutoLoad</code>	load on open
<code>RequiredField</code> / <code>ErrorMessage</code>	form <code>DataAnnotations</code>	server-validated
<code>Enabled = false</code>	<code>Disabled</code>	greys control, locks input
<code>ShowNewButton</code> / <code>NewButtonUrl</code>	<code>NewUrl</code> / <code>NewText</code>	+ opens add-record screen
<code>AfterSelectClientAction</code>	<code>CallbackName</code>	global fn run after select
<code>AfterClearClientAction</code> / <code>ValueCleared</code>	<code>OnClearJs</code>	global fn run after clear
<code>ValuesReturned(Hashtable)</code>	<code>PickFields</code> (+ <code>CallbackName</code>)	fill many fields from the row
<code>ConnectionString</code>	<code>DatabaseSelector</code> (global)	not per-control
"Go" exact resolve	<code>EnableGo</code> + <code>GoFilterKey</code>	single-field exact, by design

Legacy property / event	New	Notes
dependent scoping	DependsOn / DependsRequired / DependsMessage / ClearTargets	e.g. district scoped to state
AutoPostBack	(none — AJAX callback)	no server round-trip on select

Usage recipe (state picker, callback mode — from District)

```
@{
    var statePicker = new RecordPickerModel
    {
        Label = "State",
        ButtonText = "Select State",
        Placeholder = "State code",
        ValueName = "Form.StateId", ValueId = "Form_StateId",
        ValueValue = Model.Form.StateId > 0 ? Model.Form.StateId.ToString() : null,
        TextName = "Form.StateName", TextId = "Form_StateName",
        TextValue = Model.Form.StateName,
        Descriptor = new SearchDescriptor
        {
            Title = "States",
            DataUrl = "/Bank/District",
            Command = "searchStates",
            IdKey = "Id",
            Columns = [new SearchColumn("StateName", "State"), new SearchColumn("Code", "Code")],
            Filters = [new SearchFilter("StateName", "State")],
            GoFilterKey = "Code",           // Go resolves by code only (single-field, exact)
            SelectMode = "callback",
            PickValueKey = "Id",
            PickTextKey = "StateName",
        },
    },
};
}
<partial name="_RecordPicker" model="statePicker" />
<span asp-validation-for="Form.StateId"></span>
```

```
// Controller – the picker's command; raw id (reference data, re-validated on save)
private async Task<IActionResult> SearchStatesAsync(SearchRequest req, CancellationToken ct)
{
    var resp = await searchEndpoint.RunAsync(async (r, c) =>
    {
        r.Filters.TryGetValue("StateName", out var name); // dialog browses by name
        r.Filters.TryGetValue("Code", out var code);      // Go resolves by code
        var term = r.IsGo ? code : name;
        var res = await svc.SearchStatesAsync(term, r.Sort, r.Desc, r.Page, r.PageSize, c, exact: r.IsGo);
        return res.IsSuccess ? res.Value : ([], 0);
    }, ct);
}
```

```

    }, "Id", ScreenKey, req, ct, tokenizeId: false);
    return Json(resp);
}

```

Key differences / gotchas

- All new picker props are additive with safe defaults — set only what a screen needs.
- `EnableGo = false` (dialog-only picker, e.g. Employee) makes the box read-only; the user selects via the dialog.
- Reference-data pickers pass `tokenizeId: false` (the chosen id is re-validated on save); grids that navigate use tokens.
- Go is **single-field by design** (`GoFilterKey` , else the first filter) — never the multi-column `Q` path.
- **Keyboard (shared search dialog)**. After Search returns at least one row, focus moves into the result grid and the first row is highlighted. With the grid focused, Down / Up move the highlight (scrolling into view) and Enter selects the highlighted row (same as clicking — fills/navigates per `SelectMode`). Up clamps at the first row. Down at the last row of the page moves focus to the **"Next >"** pager button when a next page exists (the button is enabled); the highlight does **not** auto-advance — you press **Enter** on that focused Next button to load the next page, after which focus returns to the first row of the new page. (The Prev button is not auto-focused; Up stays clamped.) Esc closes. Arrows and Enter act only while the grid has focus; in the filter boxes arrow keys still move the text caret and Enter behaves as Tab, so you reach the grid by clicking Search.

C. Search Box / Lookup dialog

The browse-and-pick grid behind a Search button or a picker.

At a glance

- Legacy: `UserControls\TssiplSearchBox.ascx` + the `RecordSelector.aspx` popup grid.
- New: `Views\Shared\_SearchModal.cshtml` (one instance in the layout, id `cbs-search`) · `wwwroot\js\cbs-search.js` · models `SearchDescriptor` / `SearchColumn` / `SearchFilter` / `SearchRequest` / `SearchResponse` · service `SearchEndpoint` · infra `RowToken` .

Mapping

Legacy	New	Notes
<code>showModalDialog</code> popup window	shared in-page modal <code>#cbs-search</code>	one per layout
SQL in the URL	a fetch delegate into the screen's <code>Result</code> - returning service	server owns it
per-field search inputs	<code>SearchRequest.Filters[key]</code>	validated against <code>Columns</code>
free-text box	<code>SearchRequest.Q</code>	when <code>Filters</code> is empty
<code>uxMaxRowCount</code> + "more records"	real paging (<code>Page</code> / <code>PageSize</code>) + total + pager	proper paging

Legacy	New	Notes
column sorting	<code>SearchColumn.Sortable + Sort / Desc</code>	server-validated key
row select returns raw id	<code>RowToken</code> in the <code>IdKey</code> column	navigate or callback
open record by id	<code>SelectMode = "navigate" + NavigateTemplate</code>	<code>?row={token}</code>
fill a field	<code>SelectMode = "callback" + PickValueKey / PickTextKey</code>	picker mode
typed filter (string/number/date)	<code>SearchFilter.Type (text / number / date)</code>	native typed input
per-datatype operator (string LIKE or numeric/date =)	<code>SearchColumn.Match (contains / starts / equals)</code>	advisory metadata; the service applies it, default <code>contains</code>
<code>SearchCriteria.MaxLength</code> (per field)	<code>SearchFilter.MaxLength</code>	renders the dialog input's <code>maxlength</code>

Usage recipe (grid search, navigate mode — from District)

```
@{
    var searchDescriptor = new SearchDescriptor
    {
        Title = "Districts",
        DataUrl = "/Bank/District",
        IdKey = "Id",
        Columns = [new SearchColumn("StateName", "State"), new SearchColumn("DistrictName", "DistrictName"), new SearchColumn("DistrictId", "DistrictId")],
        Filters = [new SearchFilter("StateName", "State"), new SearchFilter("DistrictName", "DistrictName")],
        SelectMode = "navigate",
        NavigateTemplate = "/Bank/District?row={token}",
    };
}
<button type="button" class="btn btn-ghost" data-cbs-search='@Json.Serialize(searchDescriptor)'+SearchDistricts>
```

```
// Controller – POST dispatch + the searchData handler (fetch delegate, tokenized)
[HttpPost]
public Task<ActionResult> Submit(DistrictForm Form, string? command, SearchRequest search, CancellationToken ct)
{
    command switch
    {
        "searchData" => SearchDistrictsAsync(search, ct),
        "searchStates" => SearchStatesAsync(search, ct),
        "Update" => UpdateAsync(Form, ct),
        "Delete" => DeleteAsync(Form, ct),
        _ => SaveAsync(Form, ct),
    };
}
```

```
private async Task<IActionResult> SearchDistrictsAsync(SearchRequest req, CancellationToken ct)
{
    var resp = await searchEndpoint.RunAsync(async (r, c) =>
    {
        r.Filters.TryGetValue("StateName", out var sn);
        r.Filters.TryGetValue("DistrictName", out var dn);
        var res = await svc.SearchDistrictsAsync(sn ?? r.Q, dn ?? r.Q, r.Sort, r.Desc, r.Page, r.Pag
        return res.IsSuccess ? res.Value : ([], 0);
    }, "Id", ScreenKey, req, ct);
    return Json(resp);
}
```

`SearchEndpoint` has exactly one entry point — the fetch delegate above. There is no base-SELECT overload: filtering, sorting and paging live in the screen's `TfICbsServices` method (`Repository<T>` over the entity set), so no SQL is assembled in the framework and nothing is provider-specific.

Key differences / gotchas

- One overload: the fetch delegate (any `TfIOmniDb` call; `tokenizeId` optional — pass `false` for reference pickers whose id is re-validated on save).
- A filter's `Key` and a sort key should match a `Columns` key; the service decides what it honours and ignores anything else.
- On open by token use `RowToken.TryResolve`; on write use `TryConsume` (single-use) — never trust a raw id from the client.
- `SearchColumn.Match` (default `contains`) is advisory metadata only: the screen's search service owns the comparison, and by convention applies `contains` for text and `equals` for numeric/date fields.
- **Keyboard.** When Search returns at least one row the grid `tbody` (`#cbs-search-body`, made focusable with `tabIndex = -1`) takes focus and highlights the first row (`.is-selected`, an inset accent left-border). While the grid is focused Down / Up move the highlight (scrolled into view) and Enter selects it. Up clamps at the first row. Down at the last row of the page moves focus to the **"Next >"** pager button (tagged `data-cbs-page-next`) when a next page exists — `move()` focuses the button instead of clamping; the highlight does **not** auto-advance, so you press **Enter** on the focused Next button to load the next page (focus then returns to the first row of the new page). Prev is not auto-focused. Esc closes; mouse hover and click-to-select are unchanged. The Arrow/Enter handling is gated by a `gridActive()` check on `document.activeElement`, so the filter inputs keep native caret arrows and the app-wide Enter-as-Tab navigation; once the Next button is focused, native button activation handles Enter.

D. Notifications

Two tiers: server outcomes vs client guards.

At a glance

- Legacy: ad-hoc `alert()` / result `showModalDialog` popups scattered per screen.
- New: `Framework\Infrastructure\Notify.cs` + `Views\Shared\_Notification.cshtml` (blocking ack modal `#cbs-ack-modal`) for server outcomes; `wwwroot\js\site.js` `cbsNotify` (transient toast, class `cbs-client-`

`alert`) for client guards.

Mapping

Legacy	New	Notes
result popup after insert/update/delete	<code>this.NotifySuccess/Info/Warning/Error(...)</code> → ack modal	blocking; dismiss with OK / ✕ / Esc
exception shown to user	<code>NotifyError(text, ex)</code>	full exception logged with a reference id; detail shown only in Development
client-side <code>alert("pick x")</code>	<code>cbsNotify(text, cls, opts)</code> / <code>cbsNotifyError(text)</code>	transient toast, auto-dismiss
message survives redirect	TempData JSON (read-once)	shows once after post/redirect-get

Usage recipe

```
// Controller – server CRUD outcome (rendered by _Notification in the layout)
var result = await svc.SaveDistrictAsync(form.StateId, form.DistrictName, UserId, ct);
if (!result.IsSuccess)
{
    this.NotifyError(result.ErrorString); // or NotifyError("friendly", ex)
    return View(nameof(Index), new EditViewModel<DistrictForm> { Form = form });
}
this.NotifySuccess("Record saved successfully.");
return Redirect(HostUrl);
```

```
// Client guard – transient toast (no server round-trip)
if (!stateId) { cbsNotifyError("Please select a State first."); return; }
```

Key differences / gotchas

- Server outcomes must be **acknowledged** (the ack modal blocks); client guards are **transient** toasts — pick the right tier.
- `Notify` messages are read-once from TempData, so a refresh won't re-show them.
- Never put a raw exception in the user text — pass `ex` to `NotifyError`; it logs the detail and shows a reference id.

E. Menu / Layout

Application navigation + collapsible sections.

At a glance

- Legacy: `UserControls\MainMenu.ascx` (nested `CollapsiblePanelStart/End.ascx`).
- New: `Views\Shared\_MenuOverlay.cshtml` + `_MenuGrid.cshtml` · `Framework\Services\MenuService.cs`.

Mapping

Legacy	New	Notes
<code>MainMenu</code> + nested collapsible panels	<code>_MenuOverlay</code> / <code>_MenuGrid</code>	full-screen overlay
menu built in markup	<code>MenuService.LoadAsync(...)</code>	rights-filtered, server-side
legacy <code>~/x.aspx</code> link target	<code>a_Menus.NavigateURL</code> holds the new route	one DbMigrator script per cutover
panel show/hide script	overlay toggle (Esc / close)	in <code>site.js</code>

Usage recipe (cut a migrated screen's menu link over to the new route)

```
-- TflCbs.Tools.DbMigrator/Migrations/<provider>/NNNN_*.sql – one UPDATE per migrated screen.
-- Match the WHOLE legacy NavigateURL, not a LIKE fragment (g_State_Mapping.aspx is a different screen)
-- Point a row ONLY at its own screen's route – AllowedUrls is derived from these.
UPDATE dbo.A_MENUSET NavigateURL = '/Hr/District'
WHERE LOWER(REPLACE(NavigateURL, '~', '')) = '/ho/bank/g_district.aspx?';
```

Key differences / gotchas

- The menu is rights-filtered server-side; the same node URLs feed the `AllowedUrls` access set — so the one column covers both the link and access.
- Point a row only at its own new screen; pointing it elsewhere would grant that page to the row's holders (privilege escalation).
- This area is mostly framework-owned; a screen developer's only routine task is the one-line menu migration.

F. Action Buttons

The form toolbar (Save / Update / Delete / Search / Cancel).

At a glance

- Legacy: `UserControls\ActionButtons.ascx` — six configurable button slots with `CommandArgument` + server-side click events.
- New: `Views\Shared\_ActionBar.cshtml` (a `string[]` of labels) + the command-dispatch POST convention.

Mapping

Legacy	New	Notes
uxAction1..6 + CommandArgument	submit buttons <code>name="command"</code> <code>value="X"</code>	one POST handler switches on <code>command</code>
server Click events	controller Submit dispatch	<code>command switch { ... }</code>
fixed button control	<code>_ActionBar</code> (labels) or inline buttons	toolbar vs real submit buttons

Usage recipe (command-dispatch — from District)

```
@* View – submit buttons carry the command *@
<div class="action-bar">
    <button type="submit" name="command" value="Save" class="btn btn-primary">Save</button>
    <button type="submit" name="command" value="Update" class="btn btn-primary">Update</button>
    <button type="submit" name="command" value="Delete" class="btn btn-ghost"
        onclick="return confirm('Delete this district?');">Delete</button>
    <button type="button" class="btn btn-ghost" data-cbs-search="@Json.Serialize(searchDescriptor)">
    <a class="btn btn-ghost" href="/Bank/District">Cancel</a>
</div>
```

```
// Controller – single POST, dispatch on command
[HttpPost]
public Task<IActionResult> Submit(DistrictForm Form, string? command, SearchRequest search, Cancellation
    command switch
    {
        "searchData" => SearchDistrictsAsync(search, ct),
        "Update"     => UpdateAsync(Form, ct),
        "Delete"     => DeleteAsync(Form, ct),
        _            => SaveAsync(Form, ct),    // default = Save
    };
};
```

Key differences / gotchas

- `_ActionBar` renders a simple toolbar of `type="button"` buttons (a quick successor to `ActionButtons.ascx`); for real form submission, screens use explicit `type="submit"` buttons with `name="command"` as above.
- Keep one POST action per screen and branch on `command` — mirrors the legacy multi-button postback cleanly.

G. Account Picker

Account lookup behind one inline control with **three typed segment boxes** (Branch code + Head code + Account no — the legacy `Account.ascx` composite) that returns **both** the chosen account and its chart-of-account. Mode is

either `coa-account` (default — a two-stage dialog: Chart of Account → Account) or `account-only` (the dialog skips COA; accounts are scoped by an external COA field).

At a glance

- Legacy: `RetailBanking\Transaction\userControls\Account.ascx(.vb)` (composite 3-box plus Go plus Select) driven by two `UserControls\TssiplRecordSelector.ascx` (one for the Chart of Account, one for the Account number) plus the two-stage popup `RetailBanking\Transaction\userControls\AccountSelector.aspx`.
- New: `Views\Shared\_AccountPicker.cshtml` (inline control, identical look to `_RecordPicker`) · `Framework\Models\Shared\AccountPickerModel.cs` · `Views\Shared\_AccountSearchModal.cshtml` (one modal, two sequential stages) · `wwwroot\js\cbs-account-picker.js` (self-contained; does not touch `cbs-search.js`) · service `Tf1CbsServices\RetailBanking\AccountService.cs`.

Why its own control (not `_RecordPicker`): a record picker fills one value from one text box. The account control is a 3-segment composite, the lookup is sequential (pick a COA, then an account scoped to that COA), and it must fill **two** linked targets (account plus its head) from a single account row. `cbs-account-picker.js` orchestrates the segments, the two stages and the dual fill; it deliberately stays separate from `cbs-search.js`.

The three segment boxes: Branch code + Head code + Account no. Their max-lengths come from `BranchLen` / `HeadLen` / `AccountLen`, which the controller reads from `b_AccountNumberFormat` via `AccountService.GetAccountNumberFormatAsync` (defaults 4 / 5 / 8). **Go** zero-pads each segment to its width, concatenates them into the full account number, and resolves it exactly. A dialog pick splits the picked account number back into the three boxes (by the same widths) using `RowAccountNumberKey` (default `AccountNumber`).

Mapping

Legacy	New	Notes
<code>Account.ascx</code> composite 3-box plus Go plus Select	<code>_AccountPicker</code> (three typed segment boxes plus Go / Select / Clear)	one inline control, look identical to <code>_RecordPicker</code>
the 3 composite segments (Branch / Head / Account no)	<code>BranchCode*</code> / <code>HeadCode*</code> / <code>AccountNo*</code> (Name / Id / Value)	visible segment inputs
segment digit widths (from <code>b_AccountNumberFormat</code>)	<code>BranchLen</code> / <code>HeadLen</code> / <code>AccountLen</code> (defaults 4 / 5 / 8)	controller reads via <code>GetAccountNumberFormatAsync</code> ; pad on Go, split on pick
resolved account key plus head key	hidden <code>AccountValue*</code> (account id) plus <code>HeadId*</code> (head id)	the two values returned to the form
Chart-of-Account <code>TssiplRecordSelector</code> popup	stage 1 of <code>_AccountSearchModal</code> , fed by <code>CoaDescriptor</code>	COA grid (Command <code>searchAccountHeads</code>); only in <code>coa-account</code> mode
Account-No <code>TssiplRecordSelector</code> popup	stage 2 of <code>_AccountSearchModal</code> , fed by <code>AccountDescriptor</code>	Account grid (Command <code>searchAccounts</code>), revealed after a COA pick

Legacy	New	Notes
<code>AccountSelector.aspx</code> two-stage popup	the single two-stage <code>_AccountSearchModal</code>	clearable selected-COA banner between stages (Change re-opens stage 1)
dialog mode (with COA or account-only)	<code>Mode = coa-account</code> (default) or <code>account-only</code>	account-only skips the COA stage
account-only external COA field	<code>HeadSourceSel</code> (DOM selector) plus <code>HeadRequired</code> / <code>HeadRequiredMessage</code>	accounts scoped by that field; require it before Select/Go when configured
pick fills account boxes only	the 3 segments (split) plus hidden <code>AccountValue*</code> plus <code>HeadId*</code>	one account row fills both account and COA
Go = resolve full account number	<code>GoEnabled</code> plus the account <code>GoFilterKey</code> (exact)	Go pads plus concatenates the 3 segments; <code>SearchRequest.Mode</code> is <code>go</code> , server reads <code>IsGo</code> for an exact match
account scoped per-control SQL	<code>AccountService.SearchAccountsAsync</code> (branch from session, optional head)	parameterized joins, branch supplied by the caller never the client
COA list proc <code>pr_g_getAccountHead</code>	<code>AccountService.SearchAccountHeadsAsync</code>	leaf active heads, optional account-type CSV
account list proc <code>pr_b_GetAccountList</code>	<code>AccountService.SearchAccountsAsync</code>	excludes closed accounts
(server re-read for validate/display)	<code>AccountService.GetAccountAsync</code>	branch-scoped single-account read
filter key that scopes stage 2	<code>HeadDependsKey</code> (default <code>AccountHeadId</code>)	stage 2 posts this with the head id (picked COA or external field)
display-only side fields	<code>CustomerNameTarget</code> / <code>BalanceTarget</code> / <code>ChartOfAccountTarget</code>	DOM selectors filled from the picked row
head/number fields carried on the account row	<code>RowHeadIdKey</code> / <code>RowHeadNameKey</code> / <code>RowAccountNumberKey</code>	let Go fill the COA and split the number from the one account row
(new) host hook after resolve / clear	DOM events <code>cbs:account-picked</code> / <code>cbs:account-cleared</code>	bubbling <code>CustomEvent</code> on the picker root, <code>detail.accountId</code> ; additive, opt-in

Usage recipe (from Account Holding Amount — phase 1, selection only)

```
// Controller GET – seed the segment widths from b_AccountNumberFormat (defaults 4/5/8 on failure).
[HttpGet]
public async Task<IActionResult> Index(Cancellation token ct)
{
    var fmt = await svc.GetAccountNumberFormatAsync(ct);
    var vm = new AccHoldingAmountForm();
    if (fmt.IsSuccess) { vm.BranchLen = fmt.Value.Branch; vm.HeadLen = fmt.Value.Head; vm.AccountLen
```

```

    return View(vm);
}

```

```

@* View – one AccountPickerModel with BOTH descriptors; the screen owns columns + filters per stage.
Three segment boxes (Branch / Head / Account no); their widths come from the form. *@
@{
    var accountPicker = new AccountPickerModel
    {
        Label = "Account",
        DataUrl = "/RetailBanking/AccHoldingAmount",
        Mode = AccountPickerModel.ModeCoaAccount,    // or ModeAccountOnly (set HeadSourceSel below)

        BranchLen = Model.BranchLen, HeadLen = Model.HeadLen, AccountLen = Model.AccountLen,
        BranchCodeName = "BranchCode", BranchCodeId = "BranchCode", BranchCodeValue = Model.BranchCodeValue,
        HeadCodeName = "HeadCode", HeadCodeId = "HeadCode", HeadCodeValue = Model.HeadCodeValue,
        AccountNoName = "AccountNo", AccountNoId = "AccountNo", AccountNoValue = Model.AccountNoValue,

        AccountValueName = "AccountId", AccountValueId = "AccountId",
        AccountValueValue = Model.AccountId > 0 ? Model.AccountId.ToString() : null,
        HeadIdName = "AccountHeadId", HeadIdId = "AccountHeadId",
        HeadIdValue = Model.AccountHeadId > 0 ? Model.AccountHeadId.ToString() : null,

        // account-only mode (omit in coa-account mode):
        // HeadSourceSel = "#Form_AccountHeadId", HeadRequired = true,
        // HeadRequiredMessage = "Select the Chart of Account first.",

        CustomerNameTarget = "#acc-customer-name",
        BalanceTarget = "#acc-balance",
        ChartOfAccountTarget = "#acc-coa",

        CoaDescriptor = new SearchDescriptor    // stage 1 (skipped in account-only mode)
        {
            Title = "Chart of Account",
            DataUrl = "/RetailBanking/AccHoldingAmount",
            Command = "searchAccountHeads",
            IdKey = "Id",
            Columns =
            [
                new SearchColumn("HeadName", "Chart of Account"),
                new SearchColumn("AccountHeadCode", "Code"),
                new SearchColumn("AccountTypeCode", "Type"),
            ],
            Filters = [new SearchFilter("HeadName", "Name"), new SearchFilter("AccountHeadCode", "Code")],
            SelectMode = "callback", PickValueKey = "Id", PickTextKey = "HeadName",
        },
        AccountDescriptor = new SearchDescriptor    // stage 2
        {

```

```

        Title = "Account",
        DataUrl = "/RetailBanking/AccHoldingAmount",
        Command = "searchAccounts",
        IdKey = "Id",
        Columns =
        [
            new SearchColumn("AccountNumber", "Account Number"),
            new SearchColumn("CustomerName", "Customer Name"),
            new SearchColumn("Title", "Title"),
            new SearchColumn("OldAccountNumber", "Old A/c No"),
            new SearchColumn("Balance", "Balance"),
        ],
        Filters = [new SearchFilter("AccountNumber", "Account Number"), new SearchFilter("CustomerName", "Customer Name")],
        GoFilterKey = "AccountNumber",
        SelectMode = "callback", PickValueKey = "Id", PickTextKey = "AccountNumber",
    },
};
}
<partial name="_AccountPicker" model="accountPicker" />

```

```

// Controller – POST dispatch; ONE handler per stage, both consumed via the SearchEndpoint delegate
// overload with tokenizeId:false (reference data, re-validated on a future save).
[HttpPost]
public Task<IActionResult> Submit(string? command, SearchRequest search, CancellationToken ct) =>
    command switch
    {
        "searchAccountHeads" => SearchAccountHeadsAsync(search, ct),
        "searchAccounts"     => SearchAccountsAsync(search, ct),
        _                    => Task.FromResult<IActionResult>(BadRequest()),
    };

// Stage 1 – Chart of Account.
private async Task<IActionResult> SearchAccountHeadsAsync(SearchRequest req, CancellationToken ct)
{
    var resp = await searchEndpoint.RunAsync(async (r, c) =>
    {
        r.Filters.TryGetValue("HeadName", out var name);
        r.Filters.TryGetValue("AccountHeadCode", out var code);
        var res = await svc.SearchAccountHeadsAsync(name ?? r.Q, code, AccountTypeCsv, r.Sort, r.Des);
        return res.IsSuccess ? res.Value : ([], 0);
    }, "Id", ScreenKey, req, ct, tokenizeId: false);
    return Json(resp);
}

// Stage 2 / Go – Account, scoped to the chosen head plus the user's branch (from session, never the
private async Task<IActionResult> SearchAccountsAsync(SearchRequest req, CancellationToken ct)
{

```

```

var org = store.Current!.User!.OrgElementId;
var resp = await searchEndpoint.RunAsync(async (r, c) =>
{
    r.Filters.TryGetValue("AccountNumber", out var number);
    r.Filters.TryGetValue("CustomerName", out var name);
    r.Filters.TryGetValue("AccountHeadId", out var headStr);
    int? headId = int.TryParse(headStr, out var h) && h > 0 ? h : null;
    var res = await svc.SearchAccountsAsync(number ?? r.Q, name, headId, org, r.Sort, r.Desc, r.
    return res.IsSuccess ? res.Value : ([], 0);
}, "Id", ScreenKey, req, ct, tokenizeId: false);
return Json(resp);
}

```

Key differences / gotchas

- The control is now **three typed segment boxes** (Branch / Head / Account no), not one text box. Set all three `BranchCode*` / `HeadCode*` / `AccountNo*` bindings, and seed `BranchLen` / `HeadLen` / `AccountLen` from `GetAccountNumberFormatAsync` (the `b_AccountNumberFormat` widths). The earlier single `AccountText*` and the `HeadName*` / hidden `HeadCode*` bindings were removed — the visible code now lives in the segments; only hidden `AccountValue*` (account id) and `HeadId*` (head id) post as keys.
- Go pads each segment to its width and concatenates them into the full number for an exact resolve, so the segment widths must match how the account number is stored. A dialog pick splits the picked `RowAccountNumberKey` value (default `AccountNumber`) back into the three boxes by the same widths.
- `Mode : coa-account` (default) runs the two-stage dialog; `account-only` skips the COA stage and scopes accounts by the external COA field at `HeadSourceSel`. In account-only mode set `HeadRequired` (plus `HeadRequiredMessage`) to block Select/Go until that field is filled.
- One model, **two** descriptors: `CoaDescriptor` (stage 1, unused in account-only mode) and `AccountDescriptor` (stage 2). The screen owns the columns and filters for each stage — the client never gets SQL.
- Stage 2 is scoped to the head: the JS posts the head id under `HeadDependsKey` (default `AccountHeadId`) — from the picked COA in `coa-account` mode or the `HeadSourceSel` field in account-only mode — and the controller reads that filter to scope `SearchAccountsAsync`. The `AccountDescriptor.GoFilterKey` (e.g. `AccountNumber`) is the field Go resolves exactly.
- After a COA is picked the account stage is revealed but **no longer auto-fetches** — the user clicks Search. The modal's Search and Clear buttons stay aligned because the JS toggles `cbs-has-filters` on the filter bar (same fix as `cbs-search.js`).
- Go fills **both** parts from one account row, so the account row must carry the head: keep `RowHeadIdKey` / `RowHeadNameKey` / `RowAccountNumberKey` aligned with the account row's keys (defaults `AccountHeadId` / `ChartOfAccount` / `AccountNumber`, which match `SearchAccountsAsync`'s output). Go that returns many opens stage 2 pre-filtered by the number.
- Branch scope (`OrgElementId`) comes from the session in the controller — never trust a branch from the client. Rows are returned raw with `tokenizeId:false` (the chosen id is re-validated on save), same posture as the reference-data pickers.
- **Host hook events (additive, opt-in)**. When an account is resolved — via a dialog pick OR the Go button (both go through `pickAccount`) — the picker dispatches a bubbling `CustomEvent` named `cbs:account-picked` on the picker root (`[data-cbs-account-picker]`), with `detail: { accountId }` (the resolved hidden

account id, as a string). After Clear it dispatches `cbs:account-cleared` (same shape, `accountId` empty). These always fire with safe defaults — no markup, prop, or CSS changed; a screen opts in by adding a listener. Usage: a host screen listens for the event and reads `e.detail.accountId` to refresh account-specific figures. Reference consumer: the Account Holding Amount screen (`wwwroot\js\acc-holding-amount.js`) listens for `cbs:account-picked` to POST `command=accountInfo` and fill the "Amount Withheld" box, and for `cbs:account-cleared` to blank it.

- `GetAccountAsync` exists for server-side validation/display; the maker-checker workflow is deferred.
- **Keyboard (both stages).** Each stage grid (`tbody` made focusable) gets focus and highlights its first row after that stage's Search returns at least one row. With a stage grid focused, Down / Up move the highlight (scrolled into view) and Enter selects the highlighted row — Enter on a COA row reveals the Account stage, Enter on an Account row picks the account, exactly like clicking. Up clamps at the first row. Down at the last row of a stage's page moves focus to that stage's **"Next >"** pager button (tagged `data-cbs-page-next`) when a next page exists — `moveAcc()` focuses the button instead of clamping; the highlight does **not** auto-advance, so you press **Enter** on the focused Next button to load the next page (focus then returns to the first row of the new page). Prev is not auto-focused. Esc closes; hover and click-to-select are unchanged. An `activeStage()` check gates Arrow/Enter to whichever stage grid is `document.activeElement` (`moveAcc` mirrors `cbs-search.js`'s `move`), so the segment boxes and dialog filters keep native caret arrows and the global Enter-as-Tab; once a Next button is focused, native button activation handles Enter; you reach a grid by clicking that stage's Search.

Out of scope (no counterpart yet)

Not yet migrated — build with the patterns above when their screens come up:

- `AdTabControl.ascx` — tab strip with per-tab validation.
- Domain controls: `DDPO.ascx` , `ClientInfo.ascx` , `ClientAccountsInfo.ascx` , `ReceiptPayment.ascx` , `ucEmiChart.ascx` , `loanBalanceDetail.ascx` , `Alerts.ascx` , `AdReportParameters.ascx` , `PasswordPolicy.ascx` .